1 Conceptual Questions

1. (6 points) Explain the difference between tracing-based and scripting-based JIT compilation. Give a concrete example of a Python function that would work correctly with scripting but fail with tracing.

Tracing-based JIT compilation will execute the function once, record the operations into a graph, and then compile the graph. Scripting-based JIT compilation will analyze the source code and compile it ahead of execution. So the difference is not only the sequence of compile and execution, but also what the compiler can see: tracing-based JIT only sees one concrete execution path, while scripting-based JIT can see the overall code structure, including branches. Based on this, we can say tracing-based JIT compilation is fast, but limited when the code structure depends on dynamic Python control flow.

Example:

```
@jax.jit
def f(x, flag):
    if flag:
        return x * 2
    else:
        return x + 2
```

2. (6 points) Consider the following JAX function:

```python
@jax.jit
def process_batch(data, batch_size):
    num_batches = data.shape[0] // batch_size
    results = []
    for i in range(num_batches):
        batch = data[i * batch_size:(i + 1) * batch_size]
        results.append(jnp.sum(batch ** 2))
    return jnp.array(results)
```

What potential performance issue might arise with this JIT-compiled function? How would you fix it?

The performance issue is that this function uses a Python loop and a Python list inside @jax.jit. In JAX, Python control flow inside @jit is handled during tracing, so the loop is effectively traced for a fixed structure instead of being expressed as a fully array-based computation. Another issue is that repeated slicing and appending in Python prevents the computation from being written in a more vectorized form. JAX performs best when operations are written as array programs instead of Python-driven loops.

I would fix it by reshaping the input into batches and computing the batch-wise sum of squares with vectorized operations instead of Python control:

```
@jax.jit
def process_batch_fixed(data, batch_size):
    n = data.shape[0] // batch_size
    trimmed = data[: n * batch_size]
    batched = trimmed.reshape(n, batch_size, *data.shape[1:])
    return jnp.sum(batched ** 2, axis=tuple(range(1, batched.ndim)))
```

3. (6 points) The lecture mentioned that operator fusion can reduce memory traffic. For the expression y = sin(cos(x)) + exp(x), calculate:

(a) The number of GPU kernel launches and global memory reads/writes in eager mode (assume each operation reads inputs from global memory and writes outputs to global memory)

We can execute the expression step by step: t1 = cos(x), t2 = sin(t1), t3 = exp(x), y = t2 + t3

So, The number of GPU kernel launches is 4.

For memory access:

t1 = cos(x) reads once and writes once

t2 = sin(t1) reads once and writes once

t3 = exp(x) reads once and writes once

y = t2 + t3 reads twice (t2 and t3) and writes once

So global memory has 5 reads and 4 writes.

(b) The number in fused mode (one kernel, intermediate values in registers)

For fused mode, the intermediate values are stored in registers, so it only needs to launch the GPU kernel once and return the final result. So global memory will just read once (x) and write once (y).

(c) The theoretical memory bandwidth reduction factor

Total eager memory operations = 5 + 4 = 9

Total fused memory operations = 1 + 1 = 2

So the theoretical reduction factor = 9 / 2 = 4.5

4. (6 points) Why does dynamic control flow (e.g., if statements depending on tensor values) pose a challenge for JIT compilation? Explain both the JAX and PyTorch approaches to handling this.

Because JIT tries to extract a stable graph/IR and compile it, but dynamic control flow may change across executions depending on runtime values. That makes the execution path harder to represent in one fixed compiled form.

In JAX, it avoids standard Python control flow when the branch depends on runtime values. Instead, we rewrite the control flow using JAX control-flow primitives that can be represented inside the computation graph, such as jax.lax.cond. That means JAX requires the control flow to be written in a graph-friendly way.

In PyTorch, it does not rely only on pure tracing. By using TorchDynamo to capture Python execution frames, it can perform graph extraction, graph lowering, and backend code generation. Compared with JAX, this gives PyTorch better support for dynamic and imperative code.

5. (6 points) When you run torch.compile on a function for the first time, it's slower than the eager version. Why? What specific steps happen during this first call that don't happen in subsequent calls?

Because it is trying to capture the Python frame, do graph extraction and graph lowering, and

then perform backend code generation. That means it is trying to record and understand the execution and build a graph to compile. These trace/extraction/compilation steps happen during the first call and introduce extra overhead, while subsequent calls can reuse the compiled result and therefore run faster.

2 Programming: JAX JIT Deep Dive

Part 1: Measuring Compilation Overhead (10 points)

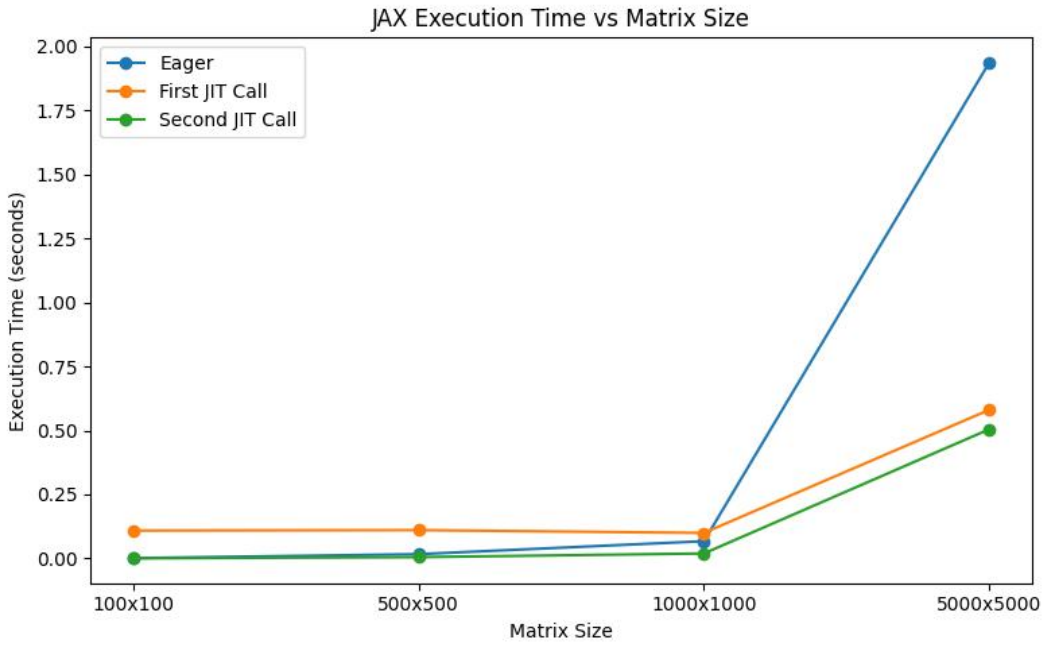
1. Create a function that performs a series of 10 elementwise operations (e.g., sin, cos, exp, log, square, etc.) on a large matrix (e.g., 5000x5000).

2. Time the execution of:

- Eager mode (without @jit)
- First call with @jit (including compilation)
- Second call with @jit (cached compilation)

3. Repeat for different matrix sizes: 100x100, 500x500, 1000x1000, 5000x5000.

4. Create a plot showing execution time vs matrix size for all three cases.



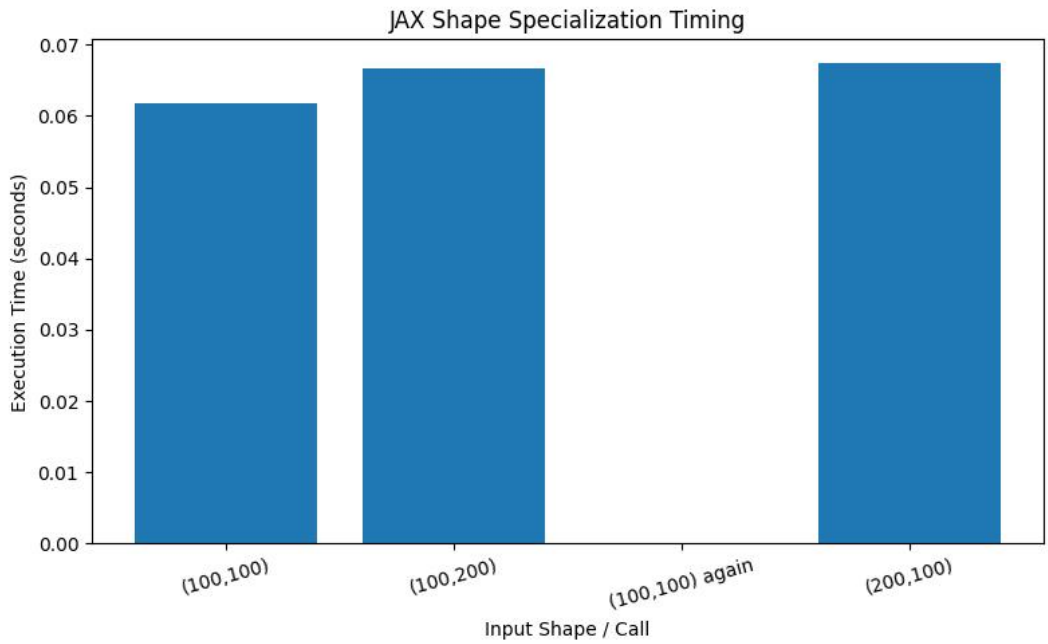
5. Explain the results: Why does the overhead vary with matrix size?

The figure shows that the first JIT call is consistently slower than the second JIT call by an extra overhead of roughly similar size. This is because the first run includes additional tracing and compilation, while later runs can reuse the compiled result and therefore avoid the same overhead.

As the matrix size increases, eager execution becomes much slower because it does not reuse a compiled and optimized result. Instead, each execution must perform the full computation and intermediate memory read/write process again. For larger matrix sizes, the growth in computation and memory traffic becomes more significant. In contrast, the one-time compilation cost of JIT is gradually amortized as the matrix becomes larger, so the actual computation time becomes the dominant factor. As a result, the advantage of JIT becomes more clear for large inputs.

Part 2: Shape Specialization (10 points)

1. Create a JIT-compiled function that computes the mean of each row in a matrix.
2. Call it with matrices of different shapes:
   - (100, 100)
   - (100, 200) - different shape
   - (100, 100) - back to original shape
   - (200, 100) - different shape again
3. Use jax.make jaxpr to inspect when retracing occurs.
4. Time each call and create a bar chart showing execution times.



5. Discuss: When does JAX retrace? What's the performance impact?

JAX stores compiled results specialized to a particular input shape and dtype, so it retraces when either the shape or dtype changes. For shapes and dtypes that have already been compiled, JAX can directly reuse the cached compiled result, so the execution time is usually much shorter. However, when the input shape changes, even if it is only a different arrangement such as (100,200) versus (200,100), JAX treats it as a new shape and performs tracing and compilation again. This introduces extra overhead, so performance can be affected when input shapes change frequently.

Part 3: Operator Fusion Analysis (15 points)

1. (5 points) Write two versions of a function that computes:
   - Version A: Write it as separate operations in a loop (eager execution)
   - Version B: Write it as a single expression and JIT compile
2. (5 points) Use NVIDIA's nvprof or PyTorch's profiler to capture kernel launches for both versions. For each version, report:
   - Number of kernels launched
   - Total execution time
   - Memory throughput

	Number of kernels launched	Total execution time	(Estimated) Memory throughput
Version A	10301	372.511ms	234.20 GB/s
Version B	1	13.371ms	0.63 GB/s

3. (5 points) Explain the results. Why does the JIT version achieve better performance? How many operations were fused together? What's the theoretical vs actual speedup?

Both implementations produced the same result (max difference = 0.0). In the version A, the profiler reported 10,301 kernel launches, while the version B launched only 1 fused kernel (loop_add_fusion). From the profiler table, the actual end-to-end speedup was about 52.2 × , while the CUDA-side speedup was about 27.9. This is because the computation is executed eagerly in Version A, , so sin, cos, and add are launched many times, which creates large kernel launch overhead and much more intermediate memory traffic. In Version B, JAX/XLA traces the whole computation and produces one fused kernel, so the overhead is much smaller. The theoretical operation count in the eager version is

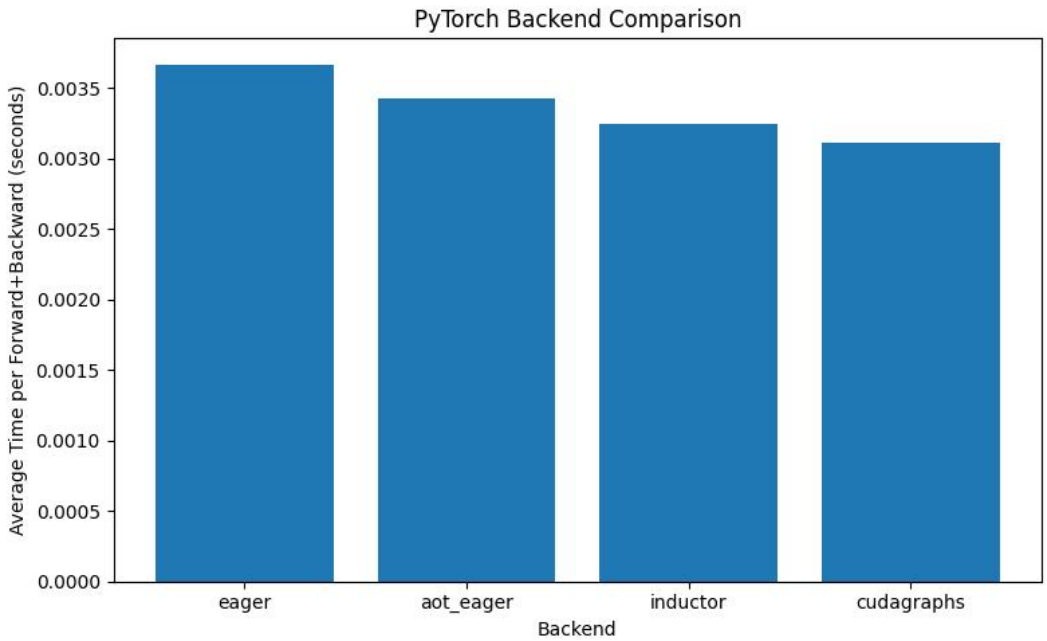
$$\sum_{i=1}^{100} (2i + 2) = 10300$$

so about 10,300 elementwise operations were fused together in practice. In the ideal case, reducing the computation from 10,301 kernel launches to 1 fused kernel suggests a theoretical speedup of about 10,301 × . However, the actual measured speedup was about 52.2 × end-to-end and about 27.9 × based on CUDA time, because real execution still includes computation cost and hardware overhead.

3 Programming: PyTorch compile Analysis

Part 1: Comparing Backends (10 points)

1. Create a simple neural network with 3 linear layers and ReLU activations.
2. Compile it using different backends
3. Run a forward + backward pass 100 times and measure average time.
4. Create a bar chart comparing performance.



5. Explain: Why do different backends have different performance? What are the tradeoffs?

Different backends perform differently because they optimize different parts of the execution pipeline. `eager` has no compile overhead but executes operations one by one. `aot_eager` captures forward and backward more systematically, but still relies on less aggressive execution. `inductor` usually performs the best because it generates optimized code for fused kernels. `cudagraphs` can reduce launch overhead for stable workloads with static shapes, but it is less flexible and may be sensitive to allocation or shape changes.

Part 2: Debugging Compilation Failures (10 points)

1. Create the following problematic functions:
2. For each function:
    - Try to compile it with torch.compile
    - Use torch. dynamo.explain() to understand why compilation fails or is suboptimal
    - Fix the function so it compiles efficiently
    - Show both the broken and fixed versions

Please check the code file to see the whole output. The three problematic functions show that torch.compile is most effective when the computation is written in a tensor-friendly way. problem1 uses a Python if based on a tensor value, so the execution path depends on runtime data and may cause graph breaks. problem2 introduces unnecessary Python container manipulation, which is not ideal for clean graph capture. problem3 uses a Python loop to build the computation step by step, so it is less efficient than a vectorized tensor expression. After

fixing them, the updated versions express the same computation more directly with tensor operations, which makes the graph cleaner and easier for the compiler to optimize.

Part 3: Graph Capture and Inspection (15 points)

1. (5 points) Write a function with the following operations:
   - Two matrix multiplications
   - A ReLU activation
   - A residual connection (add input to output)
   - A layer normalization
   - Print a Python string (e.g., “Hello”)
   - Modify a Python list (append a value)

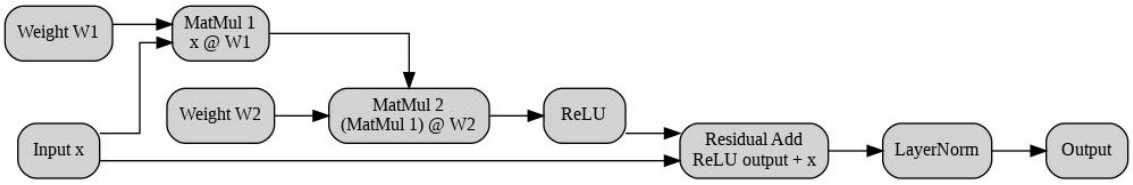
2. (5 points) Use torch.compile(..., fullgraph=True) and torch. dynamo.explain() to see what gets captured in the graph. Answer:
   - Which operations appear in the graph?

   The tensor operations appear in the graph: the two matrix multiplications, the ReLU activation, the residual addition, and the layer normalization.
   - Which operations are not captured? Why?

   The Python `print` statement and the Python list modification are not normal tensor graph operations. They are Python-side effects, so they are not represented as regular captured tensor nodes.
   - What happens to the print statement?

   With `fullgraph=True`, the `print` usually prevents full graph capture.
   - What happens to the list modification?

   The list append is also a Python-side effect. It is not part of the tensor computation graph, and under `fullgraph=True` it also tends to block full graph capture.

3. (5 points) Use make fx to see the intermediate representation:

Draw a diagram of the resulting computation graph (you can use TikZ or any drawing tool).